CMPUT 229

COMPUTER ORGANIZATION AND ARCHITECTURE I

LECTURES DELIVERED BY
ROB HACKMAN

UNIVERSITY OF ALBERTA
FALL 2026

JEFF KHUU

Introduction and Course Outline

- Different structure from previous years (past: 5 or 6 labs, labs are still take home labs but w/ o marks, there are now lab exams, the previous online quizzes are now available w/o marks)
- Some lab sessions will be mandatory to attend to complete the lab exam (completed within one hour)
- Code is written in RISC-V Assembly
- The course covers hardware/architecture, math calculation and programming in Assembly
- The midterm now only covers concepts/theory w/o a coding portion
- Hardware and the interaction with it (how computers actually work)
- What is the hardware we interact with?
- High-level chip design, performance characteristics of computers and evaluating performance (at a high level)
- Assembly programming
- Assembly programming is tedious! (but not hard)
- Canvas contains all the set of slides, course discord, former lecture videos (there as a resource but slightly outdated, available for a second perspective)
- The course follows the textbook, use it alongside the lectures

Computer Abstractions and Technology

Chapter 1

Reading includes Section 1.1 - 1.5

Theorem 1.1 (Moore's Law). *The complexity for minimum component costs has increased at a rate of roughly a factor of two per year*

Moore's Law does not typically hold up in modern times due to physical constraints. Nonetheless computers are still pervasive in modern times

There are a variety of **types** of computers

- Desktop computers (made for general purpose)
- Server computers (network-based, high capacity, performance and reliable)
- Embedded computers (hidden as components of systems, constrained by performance/ costs/power)

1.1. Below the Program

Application software is written in a high level-language (Java, C++) Below that, is the system software, a compiler that translates HLL code to machine code (as well as the operating system) Below that, is the hardware (I/O, processors, etc.)

All kinds of computers contain similar components, input which travels over memory on a "datapath" to the processor and returned as output

- Hard drive
- Processor
- Memory
- Motherboard

Definition 1.2 (Datapath). *Performs operations on data*

Definition 1.3 (Control). *Sequences datapath, memory, etc.*

Definition 1.4 (Cache memory). *On-board memory, small fast SRAM for immediate access to data*

1.2. The Big Picture

- Abstractions helps us deal with this supposed complexity in a computer
- Learning the lower level concepts make us better computer scientists however
- Instruction set architecture (ISA) is prescribed by the hardware
- Application binary interface is the ISA + System Software Interface (operating system)

1.3. *Manufacturing Integrated Circuits (ICs)*

Integrated Circuits are manufactured using silicon ingots (silicon is used because it is a semiconductor), ingots are cut into blank wafers and processed into a patterned wafer. These patterned wafers are tested and then cut into “dies” (similar to a chip)

Binary Representation

Chapter 2

As has been seen before, numbers in a computer are represented as a **binary digit** (base 2). In RISC-V a doubleword is **64 bits** (8 bytes), we number the bits 0, 1, 2, 3, … from **right to left** in a doubleword (making the least significant bit the rightmost bit)

A doubleword can represent 2^{64} combinations of 64-bit patterns and thus can represent the numbers from 0 to $2^{64} - 1$. Hardware is designed to add, subtract, multiply and divide these binary bit patterns, however if the number cannot be represented ("overflow" occurs) then it is the responsibility of the programming language and operating system to determine what is returned.

In the base 2, positional numbering system each position represents an increasing power of two

The powers of two up to 2^{10} should be memorized and it is expected to know how to convert between decimal and binary number systems.

- You can repeated division (and remainders) of two to find the binary representation of a decimal number
- Alternatively, you can convert using the largest possible power of two

The "complement" of x where x is a binary number is the flip of each bit in x denoted as $\overline{x}$

$$x = 1001_2 \Rightarrow \overline{x} = 0110_2$$

In a n-bit fix width integer, you can imagine the left-most bit to represent the negative of the largest possible power (and the rest as positive powers of two) and use that idea to sum recover the decimal representation

Naively, we can represent positive and negative numbers using a "sign bit" however this leads to several shortcomings

- it is unclear where to put the sign bit (is it the most significant?, least significant?)
- adders require an extra step to set the sign bit
- a wasted combination in the representation of $(+0, -0)$

Instead, the alternative was to use the "two's complement" representation where in general leading 0′s means positive and leading 1′s mean negative.

In "two's complement" 2_{10} is represented as `00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000010` and -2_{10} is represented as `11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110`

While "two's complement" is unable to represent one less possible positive number than negative number (i.e 2^{63} since -1_{10} is represented as the string of all 1′s) it has far more advantages than the sign bit approach

- All negative numbers have a 1 in the most significant bit (left-most bit) so negativity checks only require looking at a single bit

A useful shortcut when using "two's complement" to negate a given number is to follow the formula $\overline{x} + 1 = -x$ where $\overline{x}$ means to flip all bits in x (this naturally comes from the fact that $\overline{x} + x = -1$)

Another useful shortcut to convert from an n-bit representation to a representation with more than n bits is to fill the new more significant bits with the sign bit of the previous n-bit representation. This is called "sign extension" For example, `00000000 00000010` $= 2_{10} =$ `000000000 00000000 00000000 00000000 00000000 00000000 0000000 00000010`

Note: Two's complement gets it names from the fact that the negation/compliment of a number x is represented as $2^n - x$. Similarly a "one's complement" can be thought as just the inversion of every bit in a pattern and is represented as $2^n - x - 1$ (like converting to two's complement without the final +1 step).

In an n-bit signed number the maximum positive number is $2^{n-1} - 1$, the minimum negative number is -2^{n-1}

A final possible notation is to use "biased notation" in which the most negative value is represented as `0000 ... 0000` and the most positive value as `1111 .... 1111` with 0_{10} being represented as `10 ... 00`. The name comes from the fact that the notation can establish an arbitrary bias so the number such that the number plus the bias has a non-negative representation

In RISC-V a word has 32 bits, sometimes a 12-bit or a 20-bit "immediate value" needs to be stored

A register is able to store a single word (data needs to be loaded from the memory onto the register in order to be processed)

`t0 <- t0 + 15` The register `t0` is 32 bits but the immediate constant `15` is 12 bits (which is sign-extended to 32 bits in order to be added)

Hexadecimal Notation and Storing Data in Memory

Chapter 3

Within four bits we store the base-10 numbers 0-15, base 16 can represent these numbers with a single character. (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f)

Example `ECA8 6420` → `1110 1100 1010 1000 0110 0100 0010 0000`

$+19_{10}$ → 0000 0000 0000 0000 0000 0000 0001 0011 → 0x00000013

To store 32-bit numbers in memory we group bits into 8-bit groups called bytes (therefore 32-bit numbers are 4 bytes)

The least significant byte (is the lowest order byte, with the "least" value) similarly the most significant byte is the highest order byte

The index of each byte is called its "address", whether we store the bytes in big-endian or little-endian order is not standardized (in general)

Some places in memory do not have an address, they have a name. A register in a processor does **not** have an address, they have names. Ex. "s0" "s1" "t1" "t2" (they have numbers associated but they are NOT indexable)

In RISC-V an integer is represented by 4 bytes (u32 or i32), a memory address references a single byte in little-endian order.

```
0x10001017 0x00
0x10001016 0x00
0x10001015 0x00
0x10001014 0x0B
```

In a little-endian system the word would be `0x0000000B`, a big-endian system the word would be `0x0B000000`

Organization of a Computer & Memory Addressing

Chapter 4

A computer contains a processor and memory. The processor contains "registers", physically a register is a simple circuit that stores a number of HIGH/LOW voltages (which functionally acts as memory)

Registers store data currently being used by the processor. The processor and memory are connected by an "address bus" and "data bus". The address bus is a one-way data flow from the processor specifying the data address of interest. The data bus is a two-way data flow for transferring data

The program counter is a special register that stores the memory address of the next instruction. The instruction register contains the current instruction to execute

4.1. RISC-V

RISC-V is an open source architecture and instruction set made by Krste Asanovic and David Patterson

4.2. Arithmetic Operations

Arithmetic operations follow three-operand notation (two sources and one destination), the destination being the location to store the result, the sources acting as the operands to the function

Example:

```
add a, b, c # a <- b + c
addi a, b, 20 # a <- b + 20
sub a, b, c # a <- b - c
addi a, b, -20 # a <- b + -20 (note: no subi)
```

Immediate operands are literal constant values

Pseudo-instructions are instructions that map to one or more instructions (sometimes they may even be formally defined in the instruction set)

```
f = (g + h) - (i + j);
```

Assume `f g h`, … are all loaded into registers

```
add t0, g, h # assuming f, h, j, i, g are registers (they're not)
add t1, i, j
sub f, t0, t1
```

More accurately `f <-> s0, g <-> s1, h <-> s2, i <-> s3, j <-> s4`, this distinction is arbitrary but it is important to keep track

```
add t0, s1, s2 # t0 <- g + h
add t1, s3, s4
sub s0, t0, t1
```

Arithmetic instructions use register operands, RISC-V has a 32 x 32-bit register file (i.e 32, 32-bit registers), registers are numbered from 0 to 31 (but these are NOT easily accessed like memory) A "word" is formed by 32 bits

Registers are also given mnemonic names.

- t0, t1, … t6 for temporary values
- s0, s1, …, s11 for saved values
- Textbook uses the notation x0-x31

4.3. Addressing an Integer Array

Memory is often displayed skipping by word size. A word is the most used data unit in a processor (In 32-bit RISC-V, a word has 4 bytes (32 bits))

The `lw` (load word) instruction takes a value from memory into a specified register. The `sw` (store word) instructions take a value from a register and stores it into memory

```
A[0] = h + A[2]
```

Assuming `h <-> s2 A <-> s1`

```
lw t0, 8(s1)  # t0 <- A[2] 8 bytes into s1
add t0, t0, s2 # t0 <- h + A[2]
sw t0, 0(s1) # A[0] <- t0 (note that the first operand is the register containing data, not the destination)
```

4.4. Register vs. Memory

- Registers are faster to access than memory (no need to move through traces)
- Operating on memory data requires loading and storing (more instructions)
- Compiler will typically use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important for compilers

4.4.1. Design Principle behind RISC-V

The design principles behind RISC-V makes the common case fast (i.e small constants are common, immediate operands avoid an additional load)

4.4.2. Memory Operands

Main memory used for composite data (any l-value), words are aligned in memory (words must start on addresses that are divisible by 4 bytes)

4.5. Transferring Data between Memory and Registers

4.5.1. A `lw` execution

A third bus the "control bus" sends an instruction indicating whether the given instruction is a load/store for the data bus

For a `lw` instruction the "Read" control is sent to the memory controller, alongside the address and send the data at the memory address to the data bus for the processor register to receive.

Note instructions are also needed to be accessed from memory.

Binary Representation of Instructions

Chapter 5

Instructions are data like anything in our computer. Instructions are encoded in a binary representation called "machine code"

- RISC-V instruction representation is tried to be kept as "regular" as possible (meaning a small number of formats, applicable for many different instructions)
- An instruction is encoded as a 32-bit instruction word.

5.1. R-Type

Most arithmetic instructions of an R-Type. R-Types are represented in memory as

```
(31) funct7 | rs2 | rs1 | funct3 | rd | Opcode (0)
```

The opcode, func3 and func7 (for being 3 bits and 7 bits) tell the processor classify the instruction opcode: specifies the format (all formats contain an op-code in the lowest 7 bits) and operation of an instruction funct7/func3: together specifies the variant of an instruction to be exected rd: is the register destination operand rs1: is the first register source operand rs2: is the second register source operand

```
add t0, s7, s5
```

`t0 <-> x5 s7 <-> x23 s5 <-> x21`

5.2. I-Type

I-Type instructions represent many instructions which utilize immediates

```
(31) immediate rs1 func3 rd op (0)
```

op is 7 bits rd is 5 bits func3 is 3 bits rs1 is 5 bit immediate is stored as 12 bits (these are in the same location as the funct7 and rs2 bits in the R-type)

5.3. S-Type

```
(31) imm[11:5] rs2 rs1 func3 imm[4:0] op
```

imm[4:0] and imm[11:5] represent slices of the immediate bytes these are stored separately to keep the regularity of the instruction set

5.4. U-Type

These instructions only have destination registers and upper higher-order 20-bit immediate The remaining 12 bits of a 32-bit immediate are implicit zeros

```
imm[31:12] rd op
```

5.5. Branching

Branch statements are used for conditionals and loops (an unconditional branch is called a jump) Branch instructions will ask for a label, NOTE: labels are only used in assembly code, they do not exist outside of the code but represent an offset of the program counter to jump to.

Branching instructions utilize the SB-Type instruction.

5.6. SB-Type

```
(31) imm[12|10:5] rs2 rs1 func3 imm[4:1|11] op (0)
```

Immediate bits [10:5] are placed here because they match the S-Type instruction (similarly with bits [4:1]) The highest order bit is also always the sign bit

We stored an immediate because we must store the offset of the label. For example if the label is 3 instructions ahead (…0000 -> …000C) we need to encode the difference (12) in 13-bit binary

```
12 -> 0 0000 0000 1100
```

In 32-bit RISC V, it is viable for there to be 2 implicit zeros (since all instructions must be on addresses divisible by four), however, to increase regularity with 16-bit RISC V, only a single implicit zero

5.7. Zero

x0 is the register with the constant zero, it is hard-wired and grounded therefore cannot be overridden The constant zero is useful for a number of intructions

- moving between registers `add t2, s1, zero`
- setting a variable to 0 `add s4, zero, zero`
- performing unconditional jumps `jal zero LABEL`

5.8. Jumping

Jumping is an unconditional branching to an address. `jal` (jump-and-link) has a longer range than a branch instruction as a consequence of not needing to specify source resources

The ‘and-link’ portion of the instruction, takes a register to store the return address, `ra` is the return address register and is typically the register passed with `jal`

5.9. UJ-Type

The UJ-Type is the “unconditional jump” type that can store a 21 bit immediate (22 bit implicit)

```
(31) imm[20|10:1|11|19:12] rd op (0)
```

Bits 12-19 are stored in there position for regularity (they take the place of funct3 and rs1) The sign bit is still the most significant bit

It is uncommon for humans to need to parse the immediate value by hand, thus RISC V prefers a hardware-oriented binary format which promotes regularity

Logic Instructions

Chapter 6

`slli` (Shift Left Logical Immediate) is used to used to shift the bits in a source register `rs1` to a destination register `rd` by the immediate

For a base-2 unsigned number, a logical left shift is equivalent to multiplying the number by two (2^4 -> 2 * 2^4 -> 2^5).

Left shifts can be used for

- Quick multiplication by 2

```
slli t2, s0, k -> t2 = s0 * 2^k
```

- Moving lower half of a word to the upper half

Shifts with an immediate use a **modified** I-Type format

```
(31) func6 immediate[5:0] rs1 func3 rd op (0)
```

This modified I-Type format does not show up on the green sheet.

- A shift only needs the lower 6 bits of the I-type format's 12 bit immediate, thus the additional bits are used to classify the logical instruction

`sll` shifts the first source register `rs1` by the value stored in `rs2` and stores the result in `rd`, this is an R-Type instruction

Likewise, logical right shifts work intuitively.

6.1. A quick aside on decimal-binary conversation

In decimal,

$9,9990,099 = 10,000,000 - 10,000 + 100 - 1 = 10^7 - 10^4 + 10^2 - 10^0$

Likewise in binary

$00011111110001111100 = 2^{17} - 2^{10} + 2^7 - 2^2 = (2^6 - 1) \times 2^{10} + (2^5 - 1) \times 2^2 = ...$

which is much easier to compute.

6.2. Arithmetic Right Shift

Arithmetic right shifts (`srai`) keep the convention that shifting performs integer division by powers of two

- No arithmetic left shift
- The arithmetic will pad (to the right) by the sign-bit
- The equivalent non-immediate instruction is `sra`

6.3. Bitwise Logical Operations

`and` is a bitwise and acting on 2 registers and taking the "logical and" FOR EACH bit. Note the difference between "bitwise and" and "logical and" (for 1010_2, 0101_2 bitwise and will return 0000_2 while a "logical and" would return true)

Similarly a bitwise `or` and bitwise `xor`

There also exists immediate versions of each logical operation

`xor _, _, -1` performs a bit-flip

- two 32-bit numbers will only be zero if they are the same!

A workaround to an error in RARS is to represent negative (or hexadecimal values) as their decimal value for logical instructions Loading an immediate and shifting can perform a bit mask to similarly achieve a comparison of bits

6.4. Aside on Pseudo-instructions

`li` can load an up-to 32-byte immediate into a register

```
lui s0, 0x0FFF
li s1, 0xF00
or s0, s0, s1
```

NOTE: `addi` is harder to reason about because the immediate is SIGN-EXTENDED! (WATCH OUT FOR THIS)

Storing Arrays

Chapter 7

Recall that the following C syntax is equivalent `a[i]` to `*(a+i)`. Similarly in RISC-V assembly, the value in an array, A (with the address stored in some register) of type T will be stored at `A + sizeof(T)*i`

An example using a null-terminated string

```
add t1, s4, to # t1 <- name + k
lbu s0, 0(t1) # x <- name[k] (note lbu, not lw)
```

If the array contains 32-bit integers, i needs to be multiplied by 4 (`slli t0, t0, 2`) to correct index by the size of the data

Conditional Instructions and Loops

Chapter 8

We have learnt about different types of branch instructions `beq`, `bne`, `bgeu`, etc. as well as unconditional jumps like `jal`. We will now study basic loops and conditionals

8.1. While Loops

For the following C code, with the assumption `i := s3 j := s4`, `k := s5`, `base of save := s6` (where save is an array of 32-bit ants)

```
while(save[i] == k)
    i = i + j;
```

```
slli t1, s3, 2 # t1 <- i * 4
slli t2, s4, 2 # t2 <- j * 4
add t1, s6, t1
Loop: lw t0 0(t1)
bne t0, s5, doneLoop
add s3, s3, s4 # loop body
add t1, t1, t2 # increment
jal zero, Loop
doneLoop:
  ...
...
```

8.2. Signed vs Unsigned Comparison

`slt` and `sltu slti`, etc. always compare using signed semantics (that is, negative numbers can exist)

```
slt t0, s0, s1 #signed
```

False if -1_{10}

```
sltu t0, s0, s1 #unsigned
```

True if -1_{10} since that is $2^{32} - 1$

Note this allows us to make the comparison $0 \leq x \leq y$ very easily. In essence $x < 0 \vee x \geq y$ can be made using the instruction

```
bgeu x, y, someLabel
```

True when x is negative in two's complement and true when x is greater than the bounds of y

8.3. Basic Block

A basic block is a sequence of instructions with **no branch targets** (except at the beginning), **no embedded branches** (except at the end)

This gives us the guarantee that if the top of the block is executed, the bottom block of the sequence is also executed

Programming Example

Chapter 9

The `la` (load address) pseudo instruction allows us to retrieve the address of certain parts of a RISC-V program (labels but also specific parts of the `Data` segment of our program)

`la` is a pseudo instruction which is comprised of a `auipc` (add upper immediate program counter, which stores the program counter + immediate to a given register) and an `addi` (to the program counter)

`jalr` is an instruction which means "jump and link register", which jumps to the memory address stored in a given register

See slides for full example. Most important thing to remember is that in order to compare bitmasks, a branch instruction is required (since you can compare the bits like numbers)

Procedure Calls

Chapter 10

A **Callable Unit** is something we understand to be able to be called (perform work and then return)

- Procedure
- Function (Can be pure vs. non-pure)
- Routine
- Subroutine (This is a common nomenclature for assembly functions)
- Subprogram
- Method (Associated with Object-Oriented Programming)

A function that does not call any other functions is called a **leaf function** (think, leaf node in a tree)

Any none leaf function is a "caller" All functions are trivially "callee"s by the nature that they are a function and expect to be called.

In RISC-V there is only a single register, `ra` to store the return address, but if we have multiple non-leaf function calls, how is the value of `ra` saved before being overridden by another function call? **The stack frame**

10.1. What goes into the stack

- the value of the register `fp` (frame pointer)
- the value of the register `ra` (if the function calls another function)
- registers that are modified by the function
- local variables

The top of stack typically starts at the end of memory and moves towards lower memory addresses (the start of free memory on the stack is given by the stack pointer, `sp` is a special register that contains the ADDRESS o the top of the stack)

Before returning, non leaf functions must

1. Restore the value of `fp`, `ra`, etc.
2. Increment the stack pointer (to free the stack frame)
3. Then return to the address given by `ra`

`gp` is the global pointer, which points to the data segment for easy offset into static data

Calling Conventions

Chapter 11

All instruction sets more or less will give a number of conventions, while optional, these give a framework for code to work together (especially between developers)

These conventions usually ensure the safety of some register state. Without convention, saving the entire register file is required to ensure state is preserved

The conventions:

1. Any values that are stored in s0-s11 must be the same before and **after** calling a subroutine
2. Callee must save the values of s0-s11 (if they utilized them, leaf functions often don't)
3. Before returning, the callee must restore the value of s0-s11 (if they have been mutated)
4. Any values that are stored in t0-t6 or a0-a7 have **no** guarantee before and **after** calling a subroutine (especially the `a` registers since presumably you are getting a returned value)
    1. This means callees can use, t0-t6 and a0-a freely

11.1. Registers Used for Procedure Calls

`a0-a1` are the two registers in which to pass parameters, and for return values `a2-a7` are six arguments in which to also pass parameters If more arguments are needed to be passed, then space should be made on the stack and a pointer be passed through the `a` registers

11.2. Caller-Save vs. Callee-Save

Registers can be categorized into Caller-Save vs. Callee-Save. Caller-Save require the register to be saved by the caller if they care about the contents of the register, Callee-Save require the register to be saved by the callee if they care about the contents of the register

`ra` is caller-save, `sp` is callee-save `t0-t6` are caller-save `s0/fp` (the `x8` register) is callee-save `s1-s11` are also callee-save, `a0-a7` is caller-save (by intended design)

11.3. How is a function called?

A function cannot be executed unless it is called by another function ($\rightarrow$ every function is a callee) Every function is expected to save the `s0-s11` registers

Leaf functions do not call other functions, and can manipulate callee-save registers without having to worry about saving/restoring their original values

11.4. The need for a Frame Pointer

```
int bar() {
  int x, y;
  x = x+y;
```

```
  if(...) {
    int a[3];
    y = x+y;
  }
  x =x+y;
}
```

How much space should be allocated on the stack? We need space for `x`, `y` and also `ra`

The statement `x = x + y` requires that we load `x`, `y` from memory

```
lw t0, 4(sp) # t0 <- x
lw t1, 0(sp) # t1 <- y
add t0, t0, t1 # t0 <- x + y
sw t0, 4(sp)
```

When we reach the if-statement (and its inner scope), we allocate space on the stack for `a`. But now how would you load the values of `x` and `y`? Since the stack pointer has moved

The frame pointer always points at the beginning of a function's stack frame (most significant address)

- If we decide to use the frame pointer, both the stack pointer and the frame pointer need to be changed

```
lw t0, -4(fp) # t0 <- x
lw t1, -8(fp) # t1 <- y
add t0, t0, t1 # t0 <- x + y
sw t0, -4(fp)
```

11.5. A conventional function example

```
findLargest:
  # a0 is a pointer to an array of integers
  # a1 is the length of the array
  # a2 is an index into a comparator function
  #   takes the values to comapre in a0 and a1
  # register usage:
  # s0 still store the address of the currently largesg=1
  # s1 wil store the address of my current elem
  # s2 will store the current index
  # s3 will store length
```

```
pseudocode:
maximalElem <- a[0]
curInd <- 1
while curInd < length:
  cmp(maxialEemem
  if a0 == 0:
    maximalElem <-  a[cur]
  ++cur
)
return maximalElem
```

```
sw ra -4(sp) # store ra in the next word
sw s0 -8(sp)
sw s1 -12(sp)
sw s2 -16(sp)
sw s3 -20(sp)
addi sp, sp, -20
```

```
mv s1, a0 # a0 <- &a[0]

...

addi sp, sp, 20
lw ra -4(sp)
lw s0 -8(sp)
lw s1 -12(sp)
lw s2 -16(sp)
lw s3 -20(sp)
ret
```